

**PONTIFÍCIA UNIVERSIDADE CATÓLICA
CIÊNCIA DA COMPUTAÇÃO**

ANE M. VIANA, CAMILA C. MENEZES, GUILHERME A. ZUIM, MARIA CLARA
G. SOARES

HASKELL - APRESENTAÇÃO DO ARTIGO

BELO HORIZONTE, MINAS GERAIS
2026

Sumário

1	RESUMO DO ARTIGO	2
2	DESTAQUES IDENTIFICADOS	3
2.1	Ane Madjarian	3
2.2	Camila Menezes	4
2.3	Guilherme Zuim	5
2.4	Maria Clara Galvão	6

1 RESUMO DO ARTIGO

O artigo “Ambientes de Desenvolvimento Integrado no Apoio ao Ensino da Linguagem de Programação Haskell” discute o uso de ferramentas de desenvolvimento como suporte ao ensino da linguagem Haskell no contexto acadêmico. A linguagem, amplamente utilizada em universidades para o ensino do paradigma funcional, apresenta desafios específicos para estudantes, especialmente por diferir significativamente de linguagens imperativas tradicionais como C e Java.

Nesse cenário, o aprendizado de Haskell exige dos alunos maior capacidade de abstração, além da compreensão de conceitos como funções puras, recursão e imutabilidade. Além disso, comparativamente com outras linguagens mais populares, Haskell dispõe de um menor conjunto de ferramentas de desenvolvimento, de forma que o primeiro contato com a linguagem é geralmente através de um interpretador de comandos usado em paralelo com algum editor de programas. Para alunos habituados com ambientes de desenvolvimento integrado (do inglês, Integrated Development Environment - IDE), este modo de trabalho tende a gerar dificuldades pela falta de integração entre editor e interpretador.

Diante dessas dificuldades, o artigo propõe a utilização de Ambientes de Desenvolvimento Integrado como forma de apoiar o processo de ensino-aprendizagem. Os autores apresentam uma análise comparativa de diferentes IDEs disponíveis para Haskell, além de um estudo de caso com o uso do ambiente Eclipse em uma disciplina de graduação, com o objetivo de avaliar os impactos dessas ferramentas na aprendizagem dos alunos.

O artigo divide sua metodologia em duas partes práticas: a busca por IDEs para a linguagem Haskell e um estudo de caso para testar a ferramenta escolhida. Em um primeiro momento, um levantamento das IDEs foi feito por meio de sites Web especializados, selecionando-se 6 ambientes de desenvolvimento diferentes: Leksah, Eclipse, Winhugs, Yi, Vim e Emacs (algumas utilizando plugins para Haskell). A partir da definição de critérios de análise, os pesquisadores consultaram documentações, guias e fóruns de cada IDE para reconhecer as características que satisfaziam o que era desejável. Por fim, as IDEs foram instaladas e testadas na prática para a validação e confirmação dessas características.

Os critérios de seleção foram definidos pelos autores em conjunto com um aluno do terceiro semestre de Ciência da Computação que havia aprendido a linguagem Haskell recentemente na época. Tais critérios abrangeram requisitos técnicos (como suporte multiplataforma, compatibilidade com compiladores padrão e depurador integrado), fatores de usabilidade e edição (interface intuitiva, auto-completar, realce e checagem de sintaxe), além do suporte da comunidade (manuais completos, licença de uso livre e fóruns ativos).

Em uma análise individual das IDEs, chegou-se a algumas conclusões. O ambiente Leksah atende a muitos requisitos técnicos, de usabilidade e edição, possuindo uma comunidade bastante ativa, embora o projeto seja coordenado por apenas uma pessoa. Como característica única, possui um sistema automatizado para a correção de módulos faltantes. O Eclipse, utilizando o plugin EclipseFP, compartilha dos mesmos pontos positivos do Leksah, mas seu desenvolvimento é feito por um grupo coordenado por uma pessoa. Devido às suas características, esta ferramenta foi considerada a melhor opção para iniciantes dentre as seis avaliadas. O Winhugs atende a alguns requisitos técnicos, mas não possui suporte multiplataforma, pois é exclusivo para Windows, e oferece poucas funcionalidades de usabilidade e edição, além de ser desenvolvido por uma única

pessoa. A ferramenta Vim revelou-se complexa para iniciantes desde a instalação e também é desenvolvida por um único autor. O Emacs, também desenvolvido por uma única pessoa, apresentou dificuldades de configuração no sistema Windows. Por fim, o IDE Yi não passou pela fase de testes, pois os autores não conseguiram instalá-lo.

Com a ferramenta escolhida, um estudo de caso foi realizado com o objetivo de reunir opiniões e observações quanto ao seu uso. A pesquisa envolveu cerca de 30 alunos da disciplina de Paradigmas de Programação na Universidade Federal de Santa Maria (UFSM). Em uma primeira aula prática, os estudantes utilizaram apenas o interpretador GHC e um editor de texto à escolha. Nas práticas seguintes, totalizando cerca de 6 horas, o uso da plataforma Eclipse foi indicado. Para a coleta de dados, os autores registraram as opiniões verbais dos alunos durante as aulas e disponibilizaram um formulário online para o relato da experiência com a IDE. Como conclusão, os resultados apontaram aspectos positivos em relação à usabilidade, aos recursos integrados e ao funcionamento do ambiente. No entanto, observou-se que alguns alunos tiveram dificuldades com a instalação e configuração do programa, o que os desmotivou e os fez preferir o uso das ferramentas básicas, como um editor e um interpretador.

Para encerrar, é fundamental destacar que a pesquisa de Russi e Charão confirma que a barreira de entrada no paradigma funcional pode ser significativamente reduzida com o apoio tecnológico adequado. Ao abandonarmos o fluxo fragmentado de "editor de texto e interpretador separado", permitimos que o estudante foque no que realmente importa: a lógica e os conceitos da programação funcional. O estudo de caso demonstrou que, quando os alunos utilizam uma IDE integrada, eles se sentem mais seguros e produtivos, pois a ferramenta oferece feedbacks imediatos, como a checagem de sintaxe e a ajuda rápida, que são essenciais para quem está lidando com uma linguagem tão distinta das tradicionais C ou Java.

Além do ganho técnico, o uso de uma IDE profissional como o Eclipse atua como um catalisador motivacional dentro da sala de aula. Ao utilizar uma ferramenta que é amplamente reconhecida na indústria, o aluno deixa de ver Haskell apenas como uma curiosidade acadêmica e passa a percebê-la como uma tecnologia robusta e aplicável. Em suma, as considerações finais do artigo reforçam que, embora existam desafios de configuração inicial, os benefícios na curva de aprendizado e a satisfação dos alunos justificam amplamente a adoção dessas ferramentas no ensino superior de computação.

2 DESTAQUES IDENTIFICADOS

2.1 Ane Madjarian

Neste relatório, o ponto de maior relevância são os critérios de avaliação estabelecidos para analisar as ferramentas de apoio ao ensino de Haskell. Esses parâmetros foram definidos para identificar qual ambiente oferece a melhor experiência para quem busca produtividade, aproximando-se das exigências reais do cotidiano de um programador que prioriza a praticidade.

Os critérios que mais se destacaram por refletir essa busca por eficiência foram:

- **Instalação e Versatilidade:** A exigência de uma instalação fácil e rápida, aliada ao suporte para múltiplas plataformas (Windows, Linux e MacOS), garante que o desenvolvedor comece a produzir o quanto antes.

- Recursos de Interface e Edição: A presença de realce de sintaxe, auto-completar e múltiplas abas é vista como essencial para acelerar a escrita do código e reduzir erros manuais.
- Diagnóstico e Depuração: A inclusão de checagem de sintaxe em tempo real e de um depurador (debugger) integrado destaca a necessidade de ferramentas que ajudem a localizar falhas de lógica rapidamente, sem depender exclusivamente de ciclos manuais entre editor e interpretador.
- Ajuda Rápida e Documentação: O critério de exibir informações contextuais (como tipos de funções) ao passar o mouse reflete a necessidade de agilidade técnica, evitando interrupções constantes para consultas em manuais externos.

Ao aplicar esses critérios, o estudo identificou que o Eclipse (com o plugin EclipseFP) foi o que melhor atendeu a essas demandas de praticidade, destacando-se como a escolha mais recomendada por oferecer recursos avançados que aproximam a linguagem Haskell das ferramentas de desenvolvimento profissionais mais populares.

Mas essa classificação tem uma justificativa, o artigo destaca a dificuldade enfrentada por muitos estudantes ao ingressarem no estudo da linguagem Haskell. Como a maioria dos alunos inicia sua trajetória acadêmica com linguagens de paradigma imperativo, como C ou Java, a transição para o paradigma funcional costuma ser acompanhada de dificuldades significativas de adaptação. Nesse contexto, os autores argumentam que o uso de um Ambiente de Desenvolvimento Integrado (IDE) é fundamental, pois essa tecnologia atua como uma ferramenta de apoio capaz de minimizar erros de sintaxe e lógica, facilitando o aprendizado ao integrar recursos que tornam a experiência de programação mais fluida e menos fragmentada do que o uso isolado de editores e interpretadores.

2.2 Camila Menezes

Um dos principais destaques do artigo é a constatação de que o ensino de Haskell apresenta dificuldades adicionais quando comparado a linguagens imperativas tradicionais. Isso ocorre não apenas pelo paradigma funcional, mas também pela ausência de ferramentas integradas acessíveis no primeiro contato com a linguagem.

Nesse sentido, a análise dos critérios de avaliação das IDEs mostra-se extremamente relevante. Entre eles, destaca-se a importância da interface, que deve ser intuitiva, limpa e amigável, permitindo que o aluno foque na lógica do programa, e não na complexidade da ferramenta. Esse aspecto é fundamental no contexto educacional, pois reduz a carga cognitiva inicial e facilita o processo de aprendizagem.

Outro ponto importante abordado no artigo é a questão da instalação e configuração das ferramentas. Os autores destacam que a facilidade de instalação é um critério essencial, o que se confirma na prática. Durante o desenvolvimento do presente trabalho, observou-se dificuldade na instalação do compilador GHC, especialmente em ambientes que não utilizam sistemas baseados em Linux. Esse tipo de barreira técnica pode desmotivar alunos iniciantes e, em muitos casos, leva ao uso de compiladores online como alternativa.

Além disso, o artigo evidencia a importância de funcionalidades como checagem de sintaxe, auto-completar, realce de código e depuração integrada. Esses recursos contribuem diretamente para a produtividade e para a compreensão dos erros, permitindo que o aluno tenha um feedback mais rápido durante o desenvolvimento.

Outro destaque relevante é a integração entre editor e interpretador/compilador. A ausência dessa integração, comum no uso de ferramentas básicas, obriga o aluno a realizar múltiplas etapas manuais, como editar, salvar, carregar e executar o código, o que pode gerar confusão e erros. O uso de IDEs reduz esse problema ao centralizar todas essas funcionalidades em um único ambiente.

Por fim, o estudo de caso apresentado no artigo reforça que o uso de IDEs, especialmente o Eclipse com o plugin EclipseFP, pode contribuir positivamente para o aprendizado. Os alunos demonstraram maior facilidade no desenvolvimento e maior motivação ao utilizar uma ferramenta mais completa e semelhante às utilizadas no mercado profissional.

Dessa forma, conclui-se que a escolha de ferramentas adequadas, especialmente IDEs com boa usabilidade e integração, é um fator relevante no ensino da linguagem Haskell, podendo reduzir dificuldades iniciais e tornar o processo de aprendizagem mais eficiente.

2.3 Guilherme Zuim

O artigo de Russi e Charão aborda uma problemática central no ensino superior de computação: a alta barreira de entrada no aprendizado do paradigma funcional, especificamente através da linguagem Haskell. Os autores defendem que a transição para este paradigma é significativamente facilitada pela adoção de Ambientes de Desenvolvimento Integrado (IDEs), em contraposição ao método tradicional e muitas vezes arcaico de utilizar editores de texto simples dissociados de interpretadores de comando. A tese central é que a fragmentação das ferramentas de desenvolvimento impõe uma carga cognitiva desnecessária ao estudante, que precisa gerenciar manualmente o fluxo de trabalho enquanto tenta assimilar conceitos abstratos e complexos da linguagem.

Para fundamentar essa defesa, o trabalho apresenta uma análise comparativa detalhada entre diversas ferramentas consagradas, como Leksah, EclipseFP, Vim e Emacs. O estudo destaca que recursos como a checagem de sintaxe em tempo real e os depuradores integrados não são meros luxos tecnológicos, mas diferenciais críticos para a produtividade acadêmica. A funcionalidade de "ajuda rápida", que permite a contextualização instantânea de tipos e assinaturas de funções, é apontada como um recurso vital. Esse suporte automatizado minimiza erros primários de iniciantes — como o recorrente esquecimento de recarregar módulos modificados no interpretador — permitindo que o foco do aluno permaneça na resolução de problemas e na compreensão da lógica funcional.

No estudo de caso prático realizado na Universidade Federal de Santa Maria (UFSM), a escolha recaiu sobre o Eclipse com o plugin EclipseFP. A justificativa para essa seleção foi estratégica: a interface amigável e a familiaridade prévia que os alunos de graduação já possuíam com o ambiente Eclipse em disciplinas de Java ou C++. Essa decisão pedagógica revelou-se acertada, pois os resultados indicaram que o uso de uma IDE profissional não apenas organizou melhor o fluxo de trabalho, mas serviu como um poderoso fator motivacional. Ao utilizar uma ferramenta "de mercado", os estudantes perceberam Haskell não apenas como um exercício teórico de sala de aula, mas como parte de um ecossistema moderno e aplicável à indústria.

Por fim, a conclusão do artigo reforça que o investimento em um ambiente de desenvolvimento robusto é uma estratégia eficaz para mitigar a percepção de dificuldade intrínseca à linguagem Haskell. Embora o relato aponte desafios técnicos pontuais, como

a complexidade da instalação inicial e configuração de plugins, os benefícios a longo prazo na curva de aprendizado superam tais obstáculos. Ao aproximar as ferramentas de ensino das práticas profissionais, os autores validam que o suporte tecnológico de alto nível é capaz de promover uma evolução mais célere e sólida na compreensão dos conceitos de programação funcional, consolidando o uso de IDEs como uma tecnologia de apoio indispensável para o sucesso acadêmico na área.

2.4 Maria Clara Galvão

A partir da leitura deste artigo, acho interessante destacar como a escolha da IDE facilita a adaptação dos estudantes à linguagem Haskell. Como essa é uma linguagem que foge bastante do padrão ensinado no início dos cursos de computação, onde os alunos geralmente dão seus primeiros passos com C ou Java, o uso do Eclipse fez muita diferença. O simples fato de ser uma IDE familiar e intuitiva para quem já programa em Java ajuda a quebrar a barreira inicial com o novo paradigma. Além desse conforto na transição, outro ponto que considero essencial pontuar é como recursos específicos da ferramenta, em especial a ajuda rápida, auxiliam os desenvolvedores na prática.

Primeiramente, é notável como a transição para um paradigma totalmente diferente foi facilitada pelo uso de um ambiente já conhecido. Embora os estudantes tenham progredido no aprendizado utilizando interpretadores básicos no início, a adoção posterior do Eclipse tornou a adaptação muito mais fluida. Como programadores já tiveram algum contato com essa plataforma em algum momento para realizar projetos, especialmente para Java, a integração do plugin para Haskell eliminou a necessidade de reaprender a usar um novo software. Fazendo esse reaproveitamento do ambiente de desenvolvimento, o foco e o esforço dos alunos puderam ser direcionados integralmente para o aprendizado da lógica e da sintaxe do Haskell, comprovando que a ferramenta certa atua como um grande facilitador do ensino.

Outro ponto a destacar é como a tipagem em Haskell é trabalhada dentro deste ambiente. No artigo, é mencionado que a ajuda rápida mostra o tipo da função apontada pelo mouse, o que é algo de extrema utilidade para quem está desenvolvendo nesta linguagem, principalmente iniciantes como os estudantes da pesquisa. Como a linguagem Haskell possui uma estrutura bem diferente daquela que programadores de linguagens imperativas estão acostumados, é importante entender como as variáveis são tratadas, como os tipos se comportam e como são autor reconhecidos pela linguagem. Dessa forma, o recurso de ajuda rápida atua quase como um tutor em tempo real. Em vez de interromper o raciocínio e o fluxo de trabalho para consultar documentações externas, o desenvolvedor recebe uma dica visual na mesma hora em sua tela. Isso não apenas reduz a quantidade de erros de compilação relacionados a conflitos de tipos, mas também acelera a codificação, tornando a experiência de transição para o paradigma funcional mais rápida e produtiva.



```

17 main = do
18   writeFile "cbarr3a.svg" (reverse (processarEntrada 7890000000000))
19   , writeFile :: FilePath -> String -> IO ()

```

Figura 1: Ajuda rápida no Eclipse exibindo a tipagem da função

Fonte: Russi e Charão (2011).

Por fim, concluo que esta pesquisa foi muito interessante e importante para avaliar e propor alternativas que sejam boas para estudantes de Tecnologia da Informação. Aprender e compreender melhor as linguagens funcionais é algo que pode ajudar muito na formação de melhores programadores na área.

Referências

- [1] Davi Felipe Russi and Andrea Schwertner Charão. Ambientes de desenvolvimento integrado no apoio ao ensino da linguagem de programação haskell. *Novas Tecnologias na Educação (CINTED-UFRGS)*, 9(2), dezembro 2011.